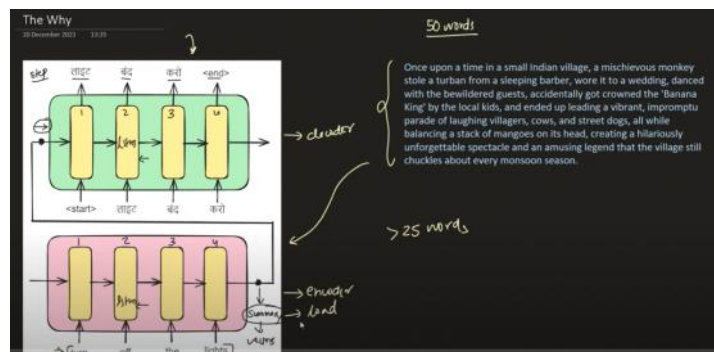


30 May 2024 07:41



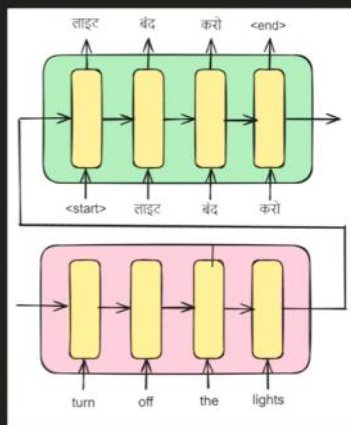
Bahdanau Attention Overview



20 December 2023 17:32

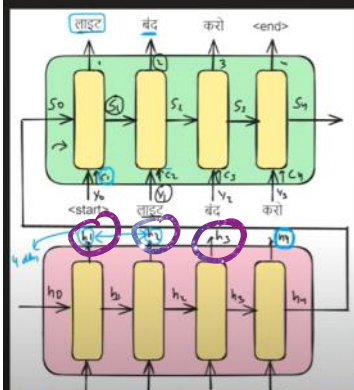


Once upon a time in a small Indian village, a mischievous monkey stole a turban from a sleeping barber, wore it to a wedding, danced with the bewildered guests, accidentally got crowned the 'Banana King' by the local kids, and ended up leading a vibrant, impromptu parade of laughing villagers, cows, and street dogs, all while balancing a stack of mangoes on its head, creating a hilariously unforgettable spectacle and an amusing legend that the village still chuckles about every monsoon season.



The What

21 December 2023 06:04



Vanilla encoder decoder

Encoder: $i \rightarrow [y_{i-1}, s_{i-1}]$

Decoder: $[y_{i-1}, s_{i-1}] + c_i \rightarrow h_i$

Attention input: $c_i \rightarrow \text{attention input}$

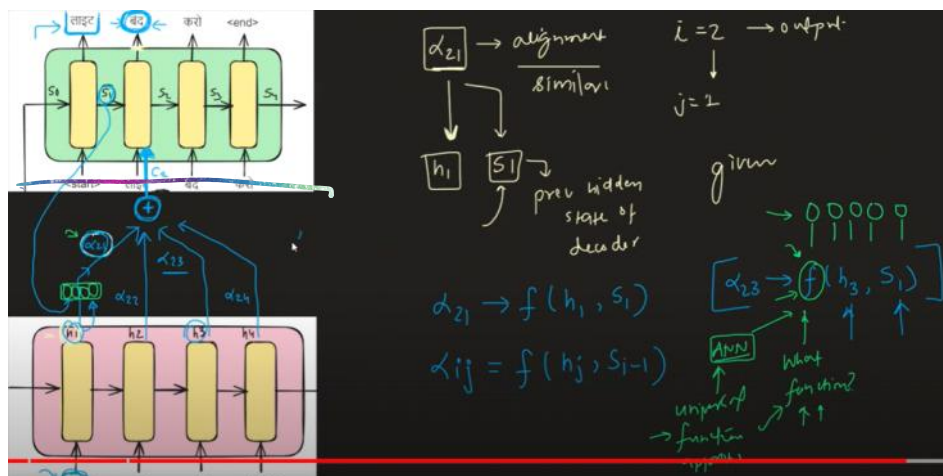
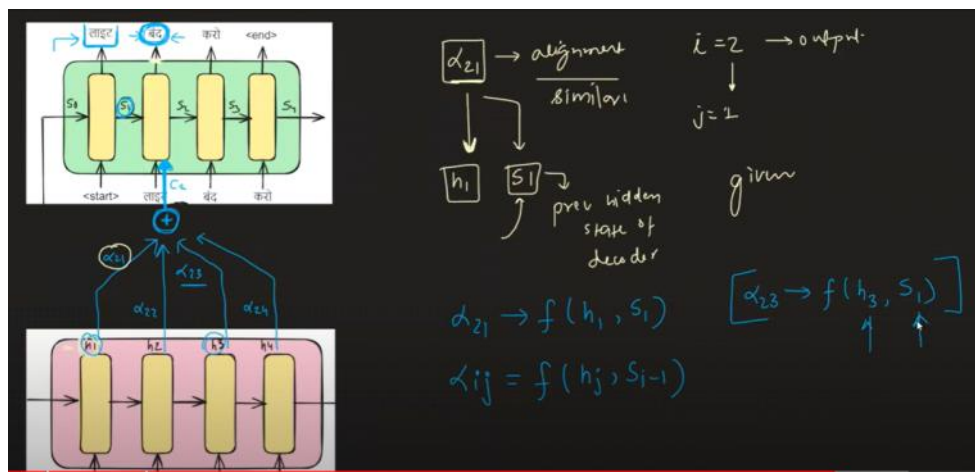
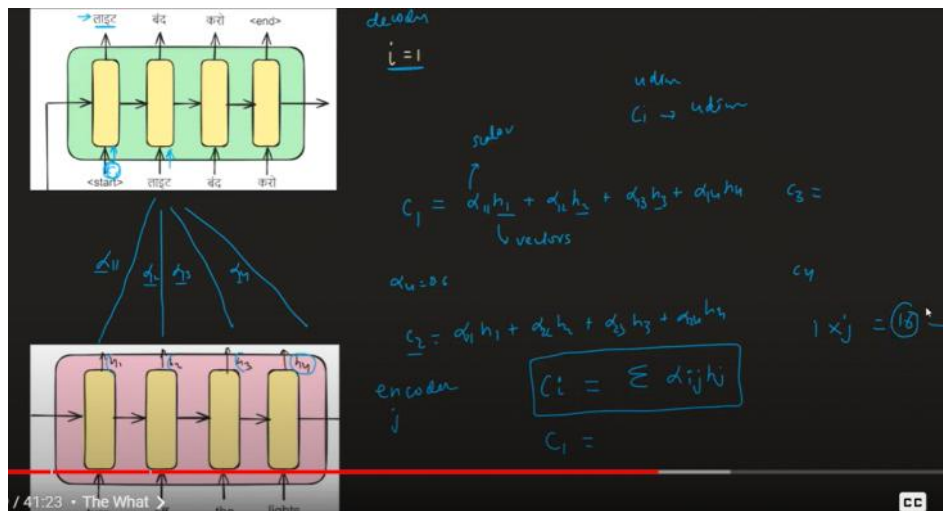
We are additional 1 more input (i.e. $h_1, h_2, \dots$) hidden state (particular timestep of the encoder) to the decoder. Which is known as C_1, C_2, C_3 , etc. So total 3 inputs are going in the decoder in the attentional model. i.e. a. S_i (the hidden state in decoder), b. each translated word, and the c. hidden state timestep (indicating which input in encoder is useful for the decoder to translate the word)
 Note: We **DO NOT** send context vector to decoder as we used to send in encoder-decoder architecture.

These C_1, C_2, C_3 are called the attention inputs

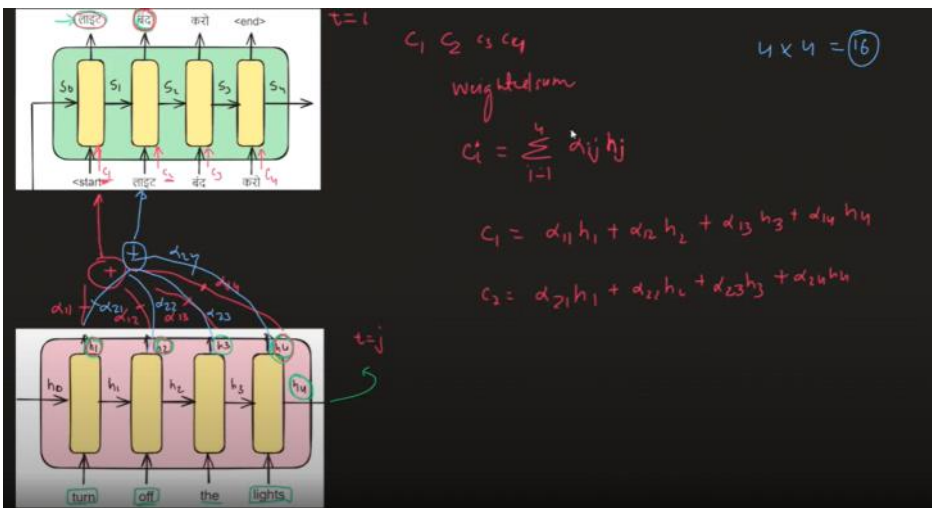
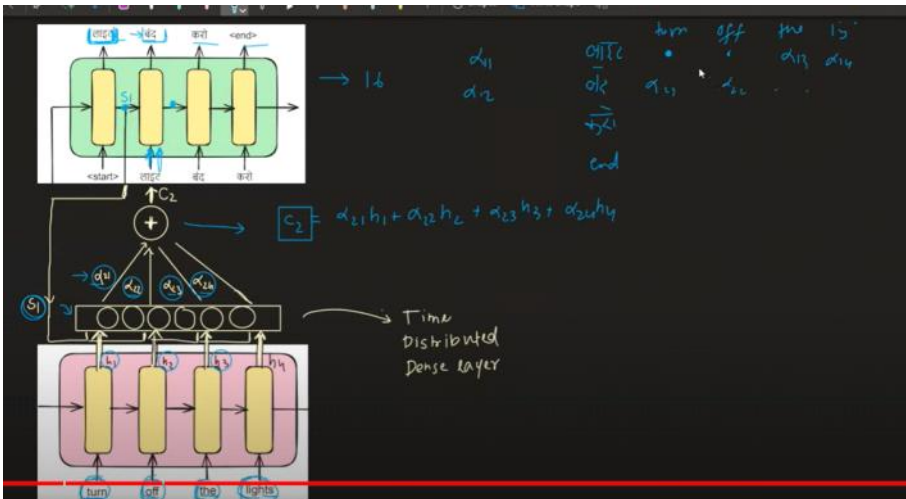
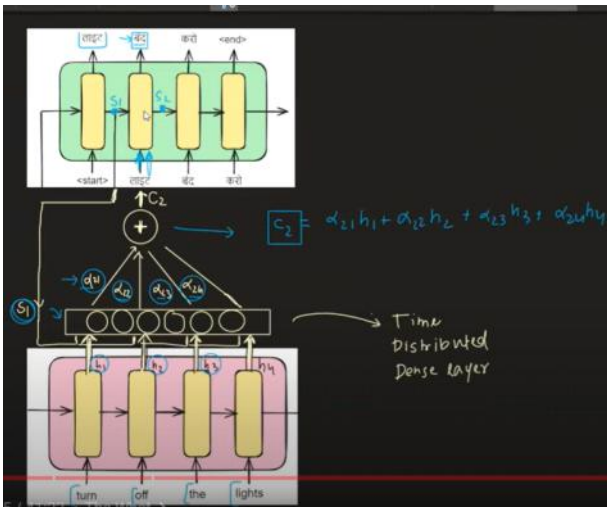
C_i job is to put the hidden state of the encoder which are important for the translation for that particular word into C_1, C_2, C_3 .

C_i is a vector and the dimension (if only 1 hidden state needs to be sent then it will have the same dimension as the hidden state, but if we need to send 2 hidden state then we need to send **weighted sum** of both the hidden states

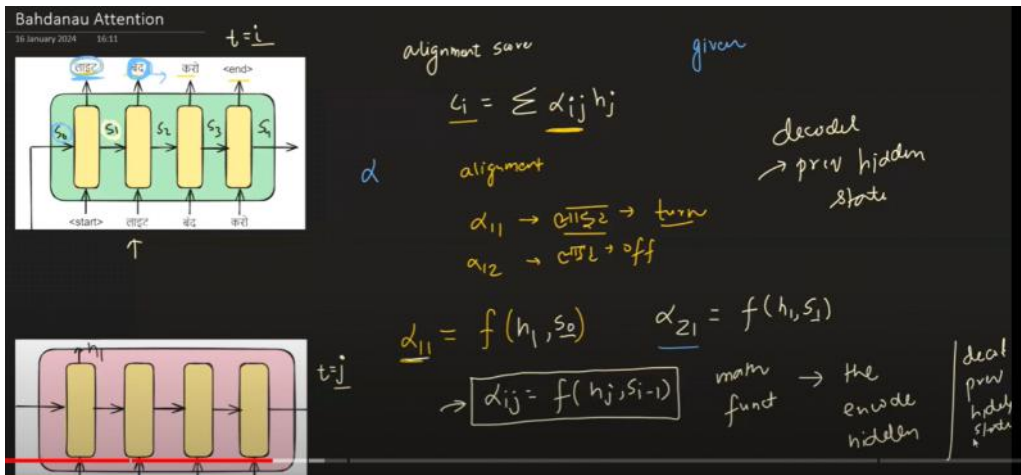
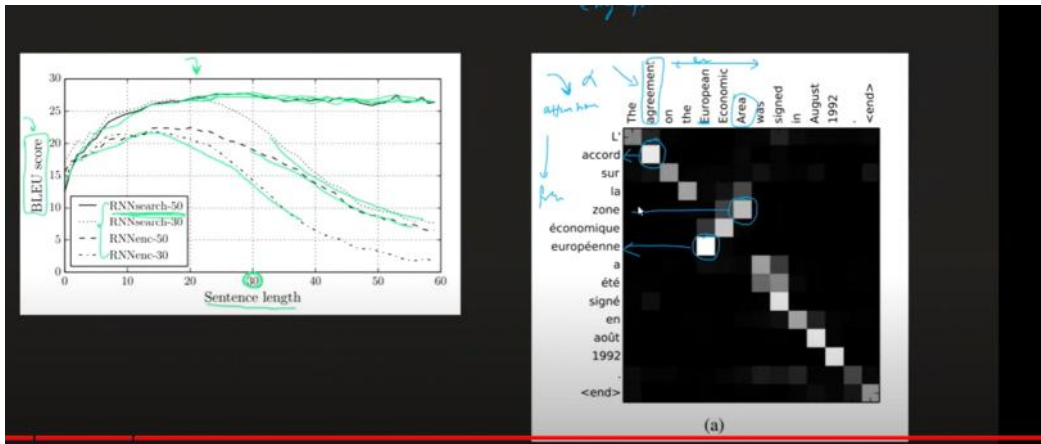
Attention mechanism will assign weights to each hidden state of the encoder. These weights are the Alpha.
 i = decoder timestep, j = encoder timestep



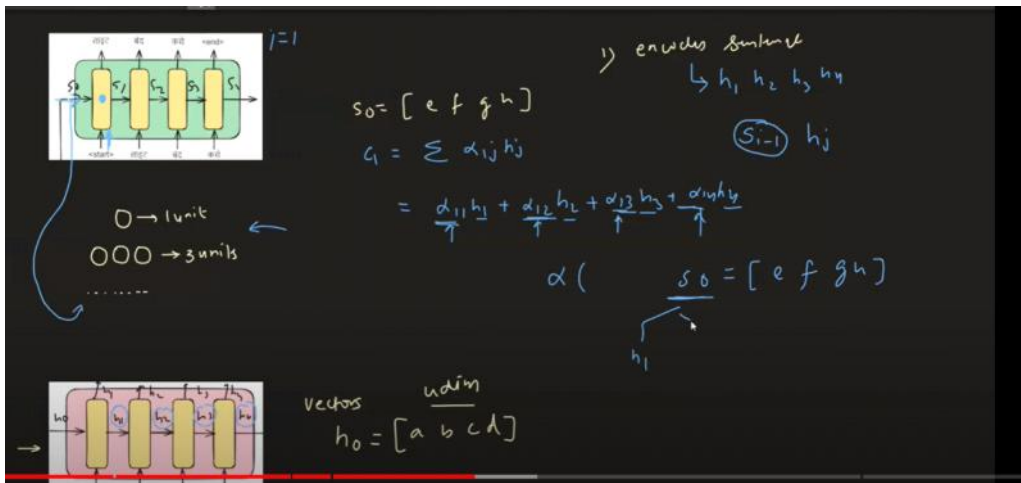
Alpha is a function of h_i and S_{i-1} and the function is nothing but a neural network which takes h_i and S_{i-1} as the input.
 Since ANN is universal function approximator, the weights and biases are getting adjusted during backpropagation. The ideal weights and biases of the ANN will act as ideal function, which will help in creating the alpha

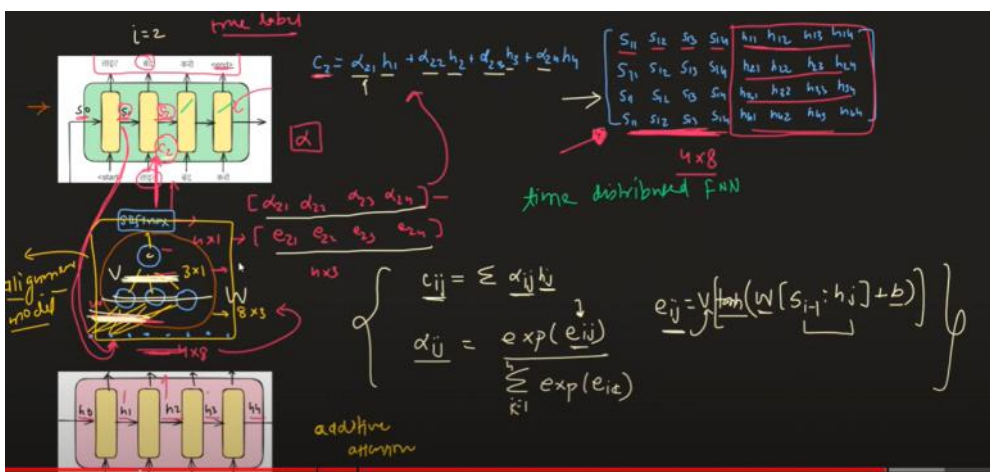
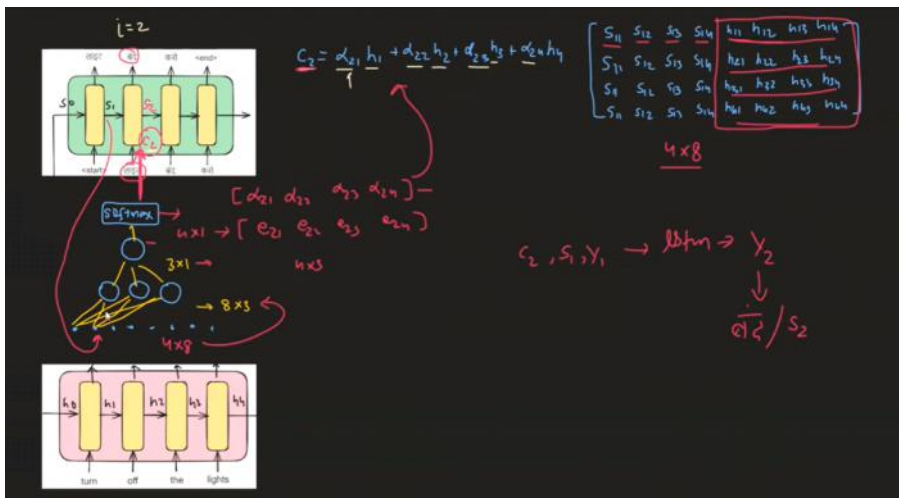
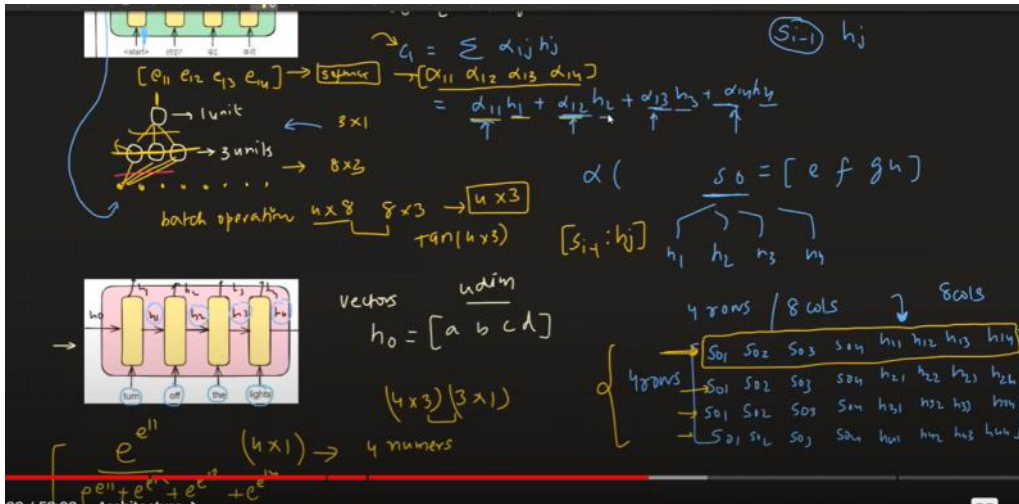
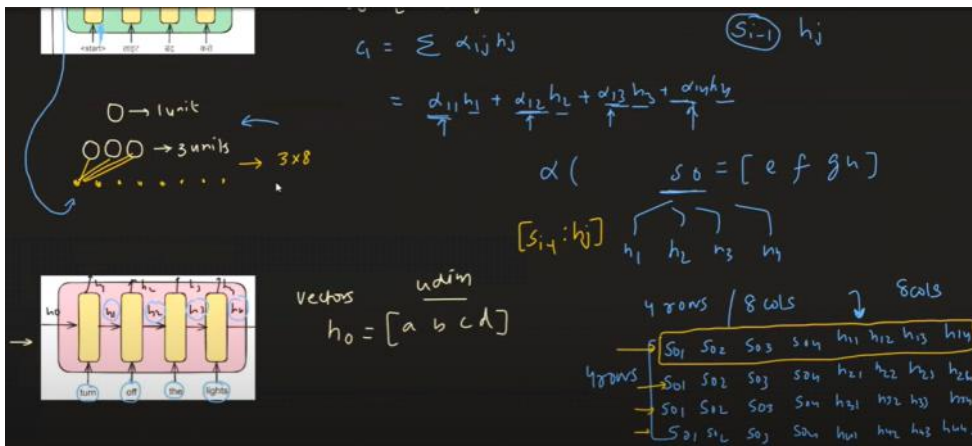


Number of Alphas = no. of words to be translated * No. of words Translated



Calculating alpha (i.e. function of H_j and S_{i-1}) - it is calculated using a feed forward ANN which generates e_{ij} and then on top of it a softmax layer which generates the alpha $_{ij}$





In Bahdanau Attention (also called as additive attention), the ANN used to calculate the Alpha is called the alignment model. The neural network is called **time distributed Fully connected neural network**

Luong Attention

17 January 2024 00:09

$$c_i = \sum \alpha_{ij} h_j \xrightarrow{\text{FFN}} [v + \tan(w[s_{i-1}; h_i] + b)]$$

$$\alpha_{ij} = f(s_{i-1}, h_j) \times$$

$$\alpha_{ij} = f(s_i, h_j) \rightarrow s_i^T \cdot h_j \rightarrow \text{dot product}$$

current ① diff

$$s_i = \begin{bmatrix} a & b & c & d \end{bmatrix}$$

$$h_j = \begin{bmatrix} e & f & g & h \end{bmatrix}$$

$$\begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} \begin{bmatrix} e & f & g & h \end{bmatrix}$$

$$[ae + bf + cg + dh]$$

$$\alpha_{ij} \leftarrow \text{softmax} \leftarrow [e_{ij}] \leftarrow \text{scaled} \rightarrow \text{attention}$$

Different from Bahdanau attention is that α_{ij} is dependent on current state of s_i and not previous state of s_i and the e_{ij} or $\alpha_{ij} = \text{function}(s_i, h_j)$. This function is just a dot product of s_i and h_j . The dot product is providing the information that which H-state of Encoder is more useful.

3rd difference as below where the C_1 is doing a dot product with S_1 in the output and not in the input and top of that we have a softmax layer to calculate α_{ij}

Luong attention (multiplicative attention) is better and computationally less expensive than Bahdanau attention (additive attention)

output

$$s_1: c_1 \rightarrow [s_1] \rightarrow \text{softmax}$$

$$s_2: c_2 \rightarrow \tilde{s}_2 \rightarrow$$

$$[e_{21} \ e_{22} \ e_{23} \ e_{24}] \text{ softmax} \rightarrow \alpha_{21} \ \alpha_{22} \ \alpha_{23} \ \alpha_{24}$$

$$\sum \alpha_{ij} h_j$$

$$\begin{bmatrix} s_1 h_1 & s_1 h_2 & s_1 h_3 & s_1 h_4 \\ e_{11} & e_{12} & e_{13} & e_{14} \end{bmatrix} \rightarrow \alpha_{11} \ \alpha_{12} \ \alpha_{13} \ \alpha_{14} \rightarrow [c_1]$$

$$c_i = \sum \alpha_{ij} h_j$$

$$\alpha_{ij} = \text{softmax}(e_{ij})$$

$$e_{ij} = s_i^T h_j$$

weights

aligning score

Luong attention

NEURAL MACHINE TRANSLATION BY JOINTLY LEARNING TO ALIGN AND TRANSLATE

Dmitry Bahdanau
Jacobs University Bremen, Germany

Kyunghyun Cho
Yoshua Bengio
Université de Montréal

attention

ABSTRACT

Neural machine translation is a recently proposed approach to machine translation. Unlike the traditional statistical machine translation, the neural machine translation aims at building a single neural network that can be jointly tuned to maximize the translation performance. The models proposed recently for neural machine translation often belong to a family of encoder-decoders and encode a source sentence into a fixed-length vector from which a decoder generates a translation. In this paper, we conjecture that the use of a fixed-length vector is a bottleneck in improving the performance of this basic encoder-decoder architecture, and propose to extend this by allowing a model to automatically (soft) search for parts of a source sentence that are relevant to producing a target word, without having to form these parts as a hard segment explicitly. With this new approach, we achieve a translation performance comparable to the existing state-of-the-art phrase-based system on the task of English-to-French translation. Furthermore, qualitative analysis reveals that the (soft) alignments found by the model agree well with our intuition.

Drawback:

1. Since LSTM were used, the input has to be sent sequentially and hence the training is slow. So we can not use large datasets

2. Transfer learning concept cannot used, i.e. this attention mechanism cannot be used as a base for Transfer modelling.
3. So, if we need to solve any task we need to build this architecture from scratch, which means we need datasets and long training time